

Министерство науки и высшего образования Российской Федерации
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ (НИ ТГУ)
Институт прикладной математики и компьютерных наук

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 1

по дисциплине «Методы верификации»

Выполнил:
студент группы № 932223
Захаров А.А.

Проверил:
Преподаватель
Лапутенко А. В.

Томск – 2026

Часть 1

По аналогии с упрощенной реализацией протокола альтернирующего бита задайте взаимодействие трех процессов. Взаимодействие между процессами должно осуществляться так:

Первый процесс отправляет любое сообщение (можно константу, бит) второму процессу;

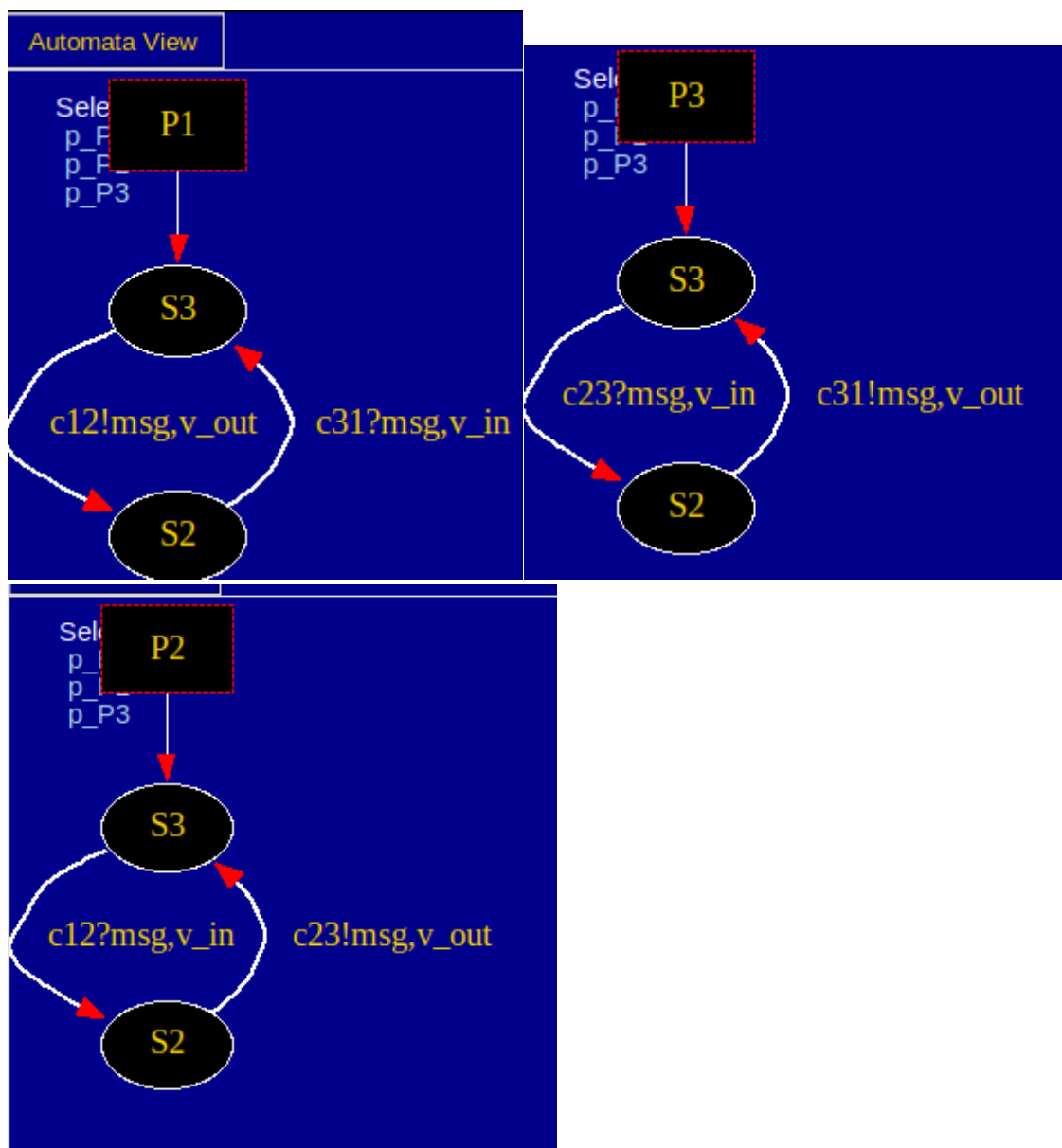
Второй процесс отправляет любое сообщение третьему процессу;

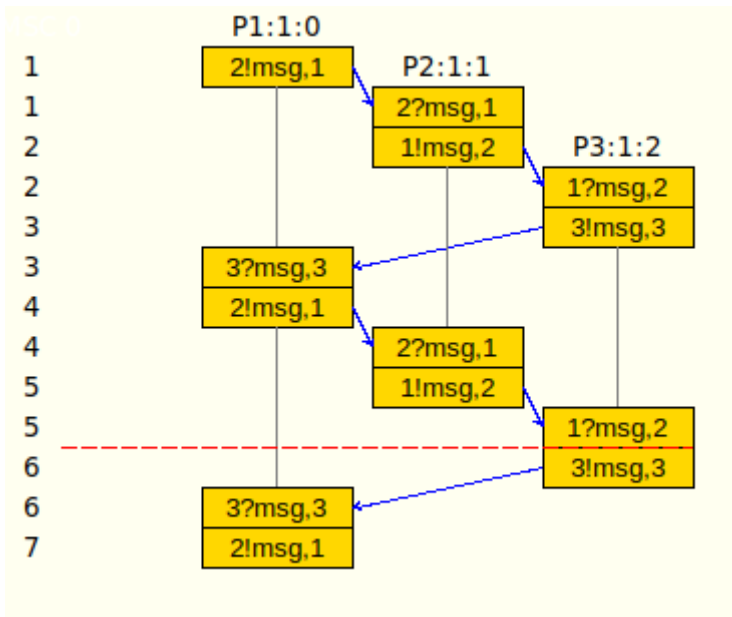
Третий процесс отправляет любое сообщение первому процессу.

Соответственно, процессы-получатели должны сообщения принимать (чтобы отправка не заблокировалась).

Запустить на выполнение в режиме симуляции (предварительно Syntax Check, если изменения – сохранить и заново Syntax Check)

Изменить программу: убрать в одном из процессов отправку сообщения. Запустить в режиме симуляции.





```

mtype = { msg };

chan c12 = [0] of { mtype, byte }; // 1 -> 2
chan c23 = [0] of { mtype, byte }; // 2 -> 3
chan c31 = [0] of { mtype, byte }; // 3 -> 1

active proctype P1()
{
    byte v_out = 1;
    byte v_in = 0;
    do
        :: c12 ! msg, v_out;    // отправляем 2-му
        c31 ? msg, v_in;        // принимаем от 3-го
    od
}

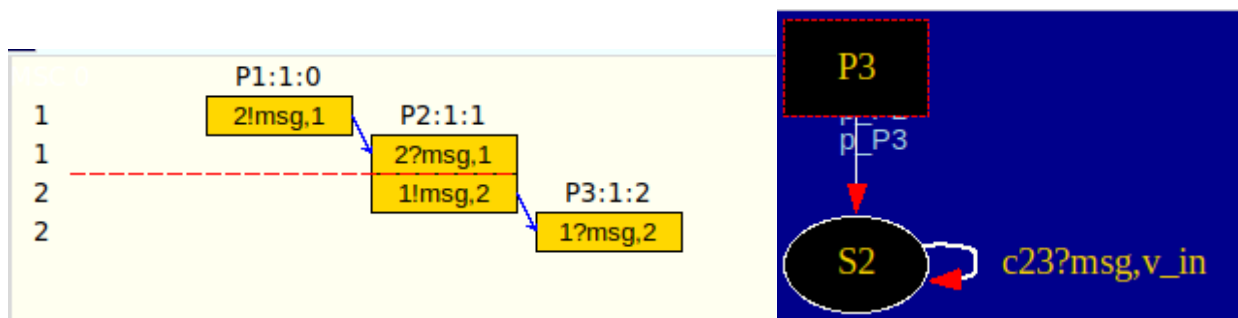
active proctype P2()
{
    byte v_out = 2;
    byte v_in = 0;

    do
        :: c12 ? msg, v_in;    // принимаем от 1-го
        c23 ! msg, v_out;      // отправляем 3-му
    od
}

active proctype P3()
{
    byte v_out = 3;
    byte v_in = 0;
    do
        :: c23 ? msg, v_in;    // принимаем от 2-го
        c31 ! msg, v_out;      // отправляем 1-му
    od
}

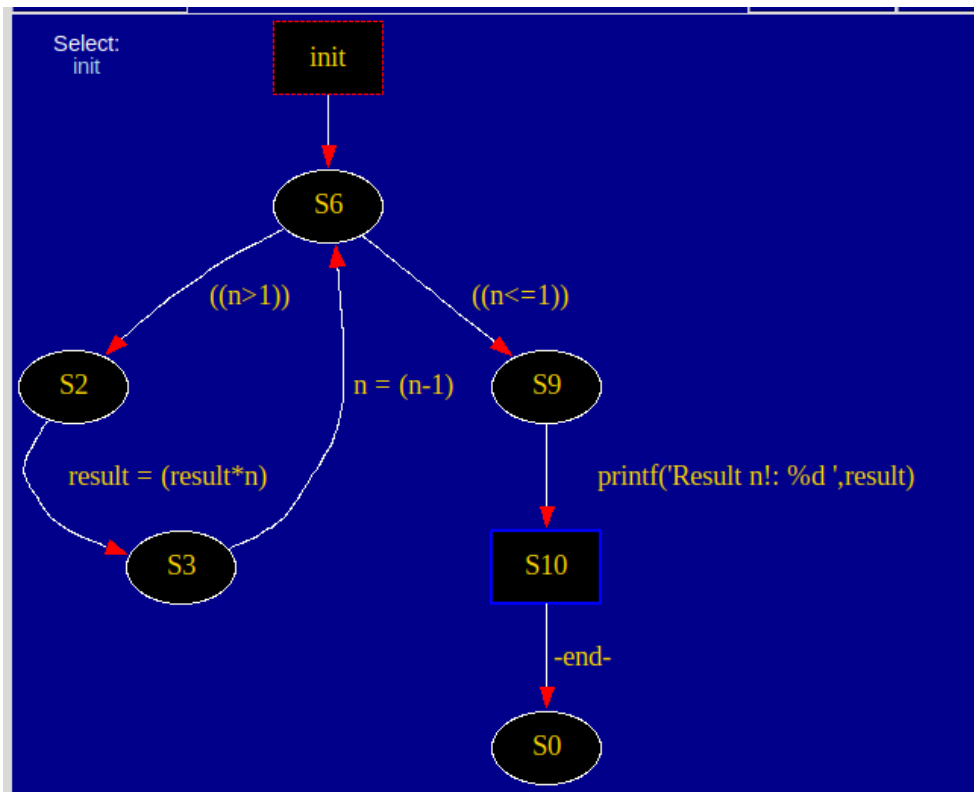
```

Убрали отправку из процесса P3:



Часть 2

1) Переписать вычисление факториала без передачи сообщения родительском процессу (вообще без использования каналов)



variable values, step 15]

```

:init:(0):n = 1
:init:(0):result = 120
  
```

```

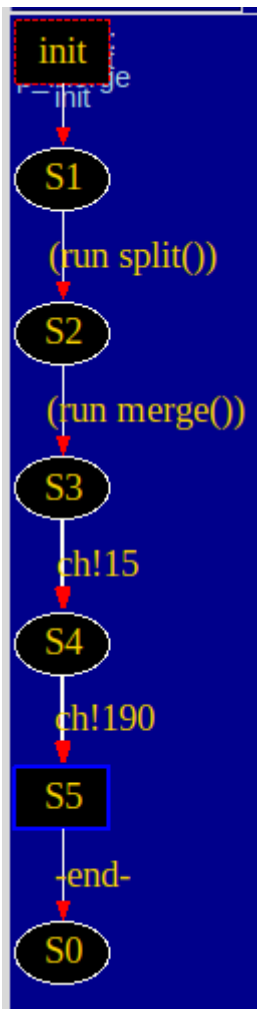
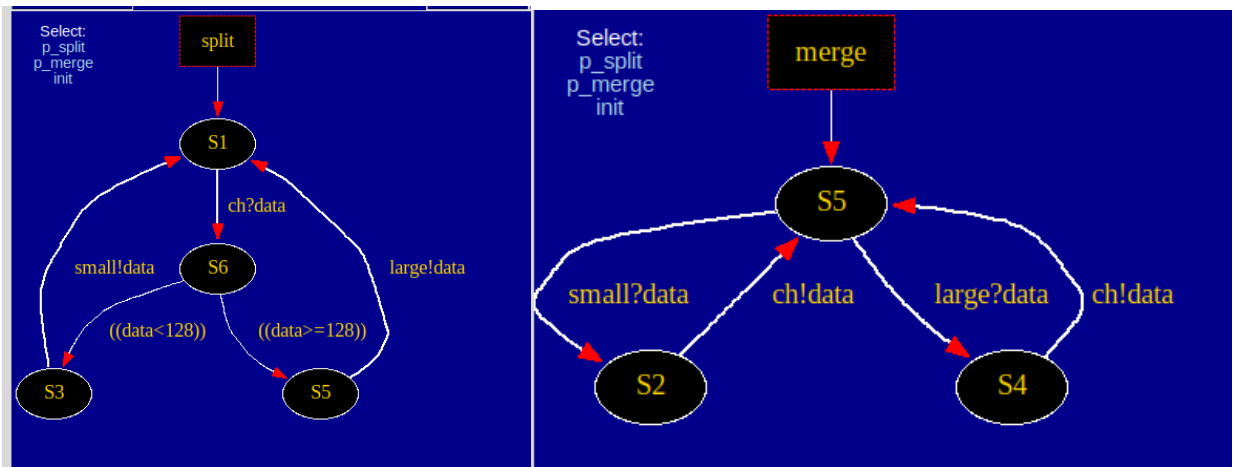
0: proc - (:root:) creates proc 0 (:init)
1: proc 0 (:init:1) factorial.pml:6 (state 1) [[[n>1]]]
2: proc 0 (:init:1) factorial.pml:7 (state 2) [result = (result*n)]
3: proc 0 (:init:1) factorial.pml:8 (state 3) [n = (n-1)]
5: proc 0 (:init:1) factorial.pml:6 (state 1) [[[n>1]]]
6: proc 0 (:init:1) factorial.pml:7 (state 2) [result = (result*n)]
7: proc 0 (:init:1) factorial.pml:8 (state 3) [n = (n-1)]
9: proc 0 (:init:1) factorial.pml:6 (state 1) [[[n>1]]]
10: proc 0 (:init:1) factorial.pml:7 (state 2) [result = (result*n)]
11: proc 0 (:init:1) factorial.pml:8 (state 3) [n = (n-1)]
13: proc 0 (:init:1) factorial.pml:6 (state 1) [[[n>1]]]
14: proc 0 (:init:1) factorial.pml:7 (state 2) [result = (result*n)]
15: proc 0 (:init:1) factorial.pml:8 (state 3) [n = (n-1)]
17: proc 0 (:init:1) factorial.pml:9 (state 4) [[[n<=1]]]
Result n!: 120
19: proc 0 (:init:1) factorial.pml:12 (state 9) [printf('Result n!: %d\n',result)]
19: proc 0 (:init:1) terminates
1 processes created
  
```

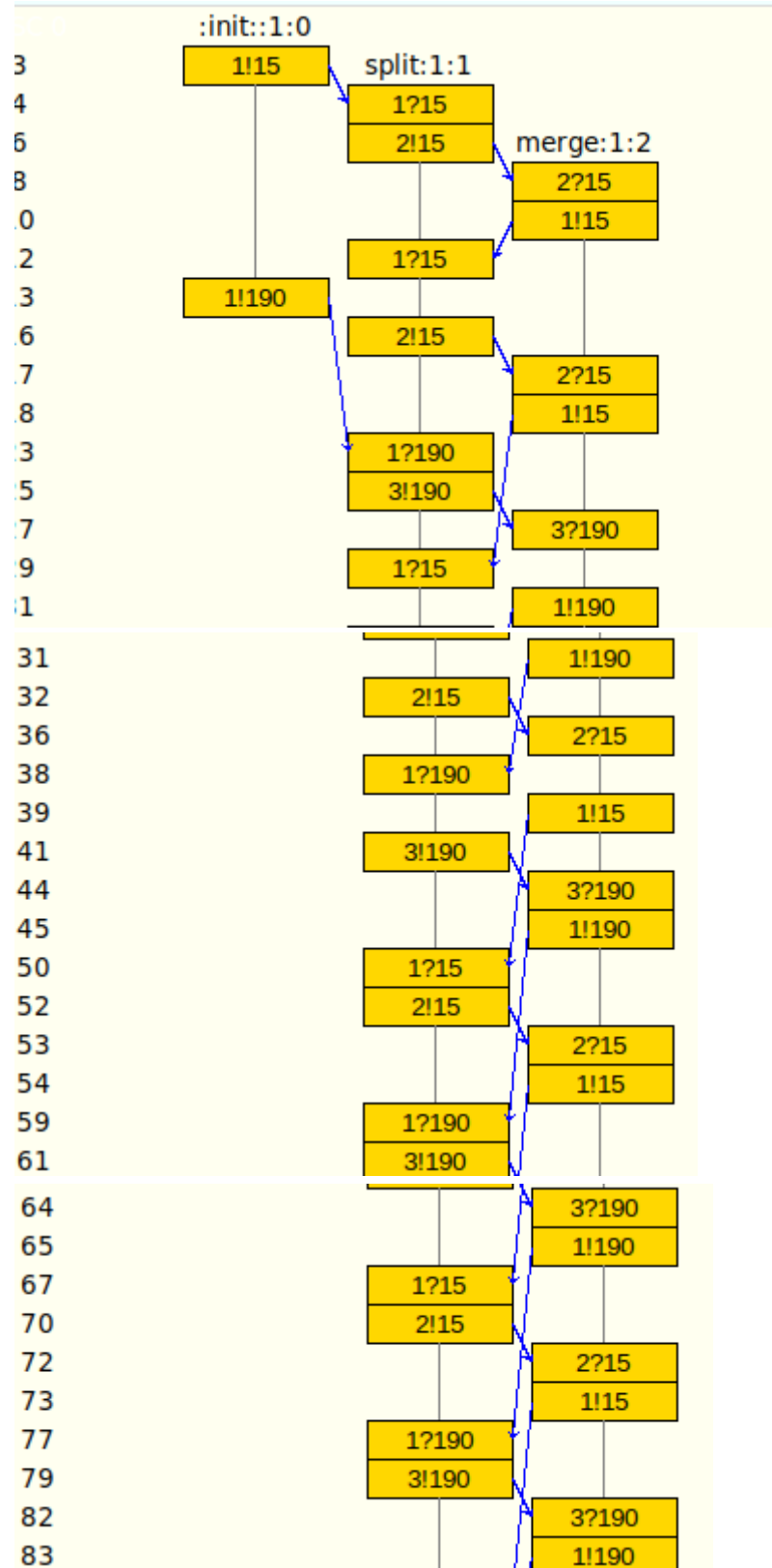
```

init
{
    int result = 1;
    int n=5;
    do
        :: n > 1 ->
            result = result * n;
            n--;
        :: n<=1 -> break;
    od;

    printf("Result n!: %d\n", result);
}
  
```

2) По полуавтомату записать процесс split(); Добавить в последний пример процесс merge(), который будет перенаправлять данные из каналов large и small обратно в канал ch; запустить merge() в init после split()





```
#define size 16

chan ch = [size] of { short };
chan large = [size] of { short };
chan small = [size] of { short };

proctype split()
{
    short data;
again:
    ch ? data;
    if
        :: data < 128 -> small ! data;
        :: data >=128 -> large ! data;
    fi;
    goto again;
}

proctype merge(){
    short data;
again:
    if
        :: small ? data -> ch ! data;
        :: large ? data -> ch ! data;
    fi;
    goto again;
}

init
{
    run split();
    run merge();
    ch ! 15;
    ch ! 190;
}
```